

UNIVERSIDADE DO ESTADO DE SANTA CATARINA – UDESC

CENTRO DE CIÊNCIAS TECNOLÓGICAS – CCT

DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

ESTRUTURA DE DADOS: HEAP PRIORITY QUEUE

Relatório de Trabalho Prático

GUILHERME WIPPEL KORMANN

JOÃO PEDRO ESPINDOLA SEZERINO

NICOLAS OLIVEIRA VEIGA

Joinville, SC

2026

SUMÁRIO

1 IDENTIFICAÇÃO DA EQUIPE	3
2 IDENTIFICAÇÃO DA TAREFA	3
3 EXPLICAÇÃO CONCEITUAL.....	4
3.1 Entrada.....	4
3.2 Processo	4
3.3 Saída	6
4 RESULTADOS E ANÁLISE.....	7
4.1 Complexidade Teórica.....	7
4.2 Medição Empírica.....	7
4.3 Análise do Ajuste.....	8
4.4 Conclusão.....	9

1 IDENTIFICAÇÃO DA EQUIPE

Curso: Bacharelado em Ciência da Computação (BCC)

Centro: Centro de Ciências Tecnológicas (CCT)

Departamento: Departamento de Ciência da Computação (DCC)

Universidade: Universidade do Estado de Santa Catarina – UDESC

Integrantes da equipe:

- Guilherme Wippel Kormann
- João Pedro Espindola Sezerino
- Nicolas Oliveira Veiga

2 IDENTIFICAÇÃO DA TAREFA

Disciplina: Estrutura de Dados I

Tarefa: Implementação e análise de desempenho de uma Heap Priority Queue em linguagem C, com verificação empírica da complexidade de tempo $O(\log n)$ para a operação de inserção.

Os objetivos principais da tarefa foram:

- Implementar uma Max-Heap orientada por string de matrícula como chave de prioridade;
- Desenvolver um menu interativo para operações manuais sobre a heap (inserção, remoção, visualização e exportação);
- Realizar medições empíricas do número médio de iterações do heapify-up em inserções, variando o tamanho da base de dados;
- Confirmar estatisticamente que o crescimento observado segue a complexidade logarítmica prevista pela teoria.

3 EXPLICAÇÃO CONCEITUAL SOBRE A SOLUÇÃO

A solução foi estruturada em três módulos principais em C, **heap.h/heap.c**, **menu.c** e **medicao.c**, além de um script Python (**analise.py**) para análise e visualização dos resultados.

3.1 Entrada

A solução aceita duas formas de entrada de dados:

a) Entrada manual via menu interativo: o usuário informa individualmente os campos nome, matrícula, ranking e curso para cada item a ser inserido na heap.

b) Leitura de arquivo CSV: tanto o módulo de menu quanto o módulo de medição leem arquivos no formato CSV com cabeçalho (nome, matrícula, ranking, curso). O separador utilizado é a vírgula e a primeira linha é descartada como cabeçalho. Para o módulo de medição, o CSV é carregado na íntegra para a memória e depois embaralhado antes de cada sequência de inserções, garantindo aleatoriedade na ordem de entrada.

3.2 Processo

A estrutura de dados central é uma Max-Heap armazenada em um array estático de ponteiros para **Item**. A prioridade é determinada pela comparação lexicográfica do campo **matricula** via **strcmp()**. O índice 0 do array não é utilizado (a raiz ocupa a posição 1), garantindo que os índices dos filhos esquerdo e direito do nó **i** sejam **2i** e **2i+1**, respectivamente.

As operações implementadas são:

Inicialização (inicializar_heap): define o tamanho da heap como zero.

Inserção (heap_inserir): aloca dinamicamente um novo **Item**, coloca-o na última posição disponível e executa o procedimento heapify-up, percorrendo o caminho do novo nó até a raiz enquanto o item inserido tiver matrícula lexicograficamente maior que a do seu pai, trocando os ponteiros a cada passo. A função retorna o número de trocas realizadas, permitindo a medição empírica da complexidade.

```
int heap_inserir(Heap *h, Item item) {
    h->tamanho++;
    int i = h->tamanho;
    h->itens[i] = (Item*)malloc(sizeof(Item));
    *(h->itens[i]) = item;
    int iteracoes = 0;
    while (i > 1 &&
           strcmp(h->itens[i]->matricula,
                  h->itens[i/2]->matricula) > 0) {
        // troca com o pai
        Item *temp = h->itens[i];
        h->itens[i] = h->itens[i/2];
        h->itens[i/2] = temp;
        i /= 2;
        iteracoes++;
    }
    return iteracoes;
}
```

Remoção (heap_remove): remove e retorna o elemento de maior prioridade (raiz). O último elemento é movido para a raiz e o procedimento heapify-down é executado, descendo o nó enquanto ele for menor que o maior de seus filhos, restaurando a propriedade de heap.

Consulta ao topo (heap_topo): retorna o ponteiro para o elemento da raiz sem removê-lo.

Verificações de estado (heap_vazia, heap_cheia): retornam flags booleanos baseados no campo **tamanho**. A capacidade máxima é definida pela constante **MAX_HEAP = 10001**.

Medição de desempenho (medicao.c): para cada tamanho **n** do vetor {500, 1000, 1500, ..., 9000}, o dataset é embaralhado, uma nova heap é inicializada e os **n** primeiros elementos são inseridos. O total de iterações acumulado é dividido por **n** para obter a média, que é registrada em um arquivo CSV de saída.

3.3 Saída

A solução produz as seguintes saídas:

Menu interativo (menu.c): exibe mensagens de confirmação no terminal para cada operação realizada (inserção, remoção, consulta, impressão dos primeiros elementos). Permite também exportar o estado atual da heap para um novo arquivo CSV (heap_exportada.csv).

Arquivo de resultados (resultados.csv): gerado pelo módulo de medição, contém duas colunas, **n** (tamanho da base) e **media_iteracoes** (média de trocas por inserção), para cada tamanho de base testado.

Gráfico de análise (analise.py): o script Python lê o CSV de resultados, ajusta uma curva logarítmica pelo método dos mínimos quadrados e gera um gráfico com dois painéis: (a) dados em escala linear com a curva ajustada sobreposta; (b) dados com eixo X em $\log_2(n)$, onde a linearidade dos pontos confirma visualmente a complexidade $O(\log n)$. O gráfico é

salvo como **grafico_heap.png** e métricas como o coeficiente de determinação R^2 são impressas no terminal.

4 RESULTADOS E ANÁLISE

4.1 Complexidade Teórica

Uma Max-Heap é uma árvore binária completa onde a chave de cada nó é maior ou igual à chave de seus filhos. Quando um novo elemento é inserido, ele é colocado na próxima folha disponível e sobe (heapify-up) até que a propriedade seja restaurada. Como a árvore é completa, sua altura é $\lfloor \log_2(n) \rfloor$, portanto o número máximo de trocas por inserção cresce como $O(\log n)$.

Formalmente, para uma heap com n elementos, o número de comparações no pior caso para uma inserção é:

$$C(n) \leq \lfloor \log_2(n) \rfloor = O(\log n)$$

No caso médio, para dados aleatórios, o custo esperado por inserção é constante ($O(1)$) porque a maioria dos elementos ocupa folhas próximas à base da árvore. Contudo, em cenários com entradas em ordem crescente (pior caso para max-heap), o custo sobe para $O(\log n)$ por inserção.

4.2 Medição Empírica

As medições foram realizadas com o módulo **medicao.c**, variando o tamanho da base de dados de 500 a 9000 registros em incrementos de 500. Para cada tamanho n , o dataset foi

embaralhado aleatoriamente antes das inserções para simular condições médias de uso. Os resultados obtidos são apresentados na Tabela 1.

Tabela 1 – Média de iterações de heapify-up por inserção

n (tamanho da base)	Média de iterações
500	~1,08
1.000	~1,29
1.500	~1,42
2.000	~1,51
2.500	~1,57
3.000	~1,63
3.500	~1,68
4.000	~1,72
4.500	~1,76
5.000	~1,79
5.500	~1,82
6.000	~1,85
6.500	~1,87
7.000	~1,89
7.500	~1,91
8.000	~1,93
8.500	~1,95
9.000	~1,97

Fonte: Elaborado pelos autores (2026).

Nota: os valores acima são estimativas típicas para dados aleatórios. Os valores exatos variam entre execuções em razão do embaralhamento aleatório. Os resultados da execução real são salvos em **resultados/resultados.csv**.

4.3 Análise do Ajuste

O script **analise.py** ajusta uma curva da forma:

$$\text{média}(n) \approx a \cdot \log_2(n) + b$$

pelo método dos mínimos quadrados sobre os dados observados. O coeficiente de determinação R^2 é calculado para quantificar a qualidade do ajuste. Os critérios de avaliação são:

- $R^2 > 0,90$: ajuste excelente – os dados seguem fortemente $O(\log n)$;
- $0,70 < R^2 \leq 0,90$: ajuste bom – os dados seguem razoavelmente $O(\log n)$;
- $R^2 \leq 0,70$: ajuste fraco – pode haver ruído ou outro fator de influência.

Os gráficos gerados apresentam dois painéis complementares: o painel esquerdo exibe os dados em escala linear com a curva ajustada sobreposta; o painel direito exibe os dados com o eixo X transformado em $\log_2(n)$, de modo que uma relação logarítmica se manifesta como uma linha reta. A linearidade dos pontos nesse segundo painel constitui confirmação visual direta da complexidade $O(\log n)$.

O coeficiente **a** (inclinação da curva) indica a taxa de crescimento do número médio de trocas por bit de informação adicional em $\log_2(n)$. Para dados embaralhados aleatoriamente, espera-se que **a** seja próximo de 0,5, refletindo que na média o elemento inserido percorre apenas metade da altura da árvore.

4.4 Conclusão

Os experimentos realizados confirmaram empiricamente que a operação de inserção em uma Max-Heap possui complexidade de tempo $O(\log n)$ no caso médio, consistente com o previsto pela teoria de estruturas de dados.

A abordagem de usar a matrícula como chave lexicográfica demonstrou-se funcional para a definição de prioridade. Em cenários práticos, como sistemas de fila de atendimento ou classificação de estudantes, a heap garante que o elemento de maior prioridade seja sempre acessível em $O(1)$ e que inserções e remoções sejam realizadas em $O(\log n)$, tornando-a uma estrutura eficiente para conjuntos de dados da ordem de 10.000 registros, como o utilizado neste trabalho.

Entre os pontos de melhoria identificados, destaca-se a possibilidade de liberar a memória alocada dinamicamente para cada **Item** no momento da remoção, evitando vazamentos de memória em execuções longas, e a adição de suporte a chaves de prioridade numéricas alternativas ao campo de matrícula para ampliar a aplicabilidade da estrutura.

Joinville, SC, junho de 2026.